

Przedmioty – Katalog Rzeczy

Dokumentacja Product Lead & UX

Wygenerowana na podstawie kodu źródłowego | 8.06.2026

packageId: pl.edu.przedmioty | minSdk: Android 6.0 (API 23) | targetSdk: Android 16 (API 36) | v1.0 (versionCode 1)

1. Dlaczego aplikacja wymaga smartfona

Aplikacja Przedmioty jest z natury aplikacją mobilną. Poniższe argumenty wynikają bezpośrednio z zaimplementowanych funkcji w kodzie źródłowym:

1.1 Aparat jako jedyna metoda wprowadzenia zdjęcia

W kodzie (ItemFormScreen.kt) jedyną metodą dodania zdjęcia jest `ActivityResultContracts.TakePicture()` – aparat fizyczny telefonu. Aplikacja nie obsługuje wyboru z galerii (brak `GetContent`). Użytkownik fotografuje przedmiot w miejscu, w którym się on znajduje – w piwnicy, garażu, na strychu. Desktop nie oferuje tej możliwości bez dodatkowego sprzętu.

1.2 Uprawnienie CAMERA jako funkcja krytyczna

`AndroidManifest.xml` deklaruje `<uses-permission android:name="android.permission.CAMERA" />`. Formularz dodawania wymaga zdjęcia – bez aparatu mobilnego podstawowy flow aplikacji jest niemożliwy.

1.3 FileProvider – bezpieczne przekazywanie URI pliku

Aplikacja używa `FileProvider` (skonfigurowanego w `AndroidManifest.xml`) do bezpiecznego przekazywania URI tymczasowego pliku zdjęcia do systemowego Intent aparatu. Jest to wzorzec wyłącznie mobilny, niemożliwy do replikacji na desktopie.

1.4 Dotykowy interfejs i natywne komponenty Material3

UI zbudowany w Jetpack Compose z Material3 (`Card`, `FloatingActionButton`, `OutlinedTextField`) jest zoptymalizowany pod dotyk: karty klikalne (`Modifier.clickable`), FAB (+) w prawym dolnym rogu, natywne dialogi (`AlertDialog`). Interakcje są naturalne na smartfonie.

1.5 Kompresja zdjęć dostosowana do możliwości urządzenia mobilnego

`ImageUtils.kt` stosuje dynamiczny `sampleSize`, skalowanie do max 640px i JPEG quality 52 – algorytm dostosowany do wydajności procesorów mobilnych i ograniczeń transferu danych Firestore (max ~650KB base64 per dokument).

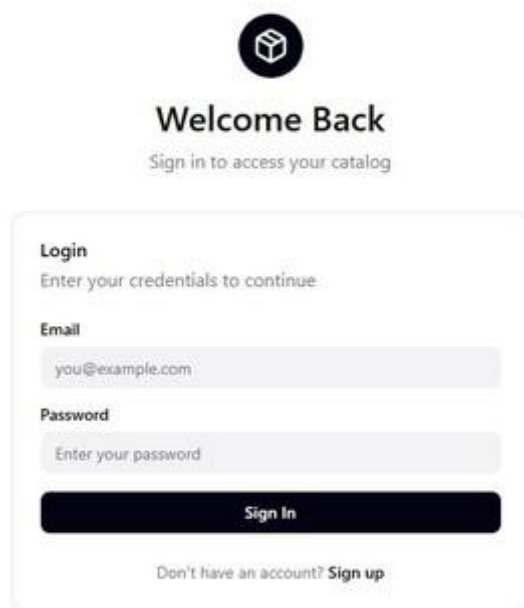
1.6 Synchronizacja w tle z Firebase

`addSnapshotListener` Firestore działa w tle Android runtime. Nowe wpisy z innych urządzeń pojawiają się automatycznie na liście bez żadnej akcji użytkownika – funkcja charakterystyczna dla aplikacji mobilnych z trwałym połączeniem sieciowym.

2. Prototyp UI – Przegląd Ekranów

Prototyp aplikacji Przedmioty obejmuje 4 główne ekrany realizujące pełny przepływ użytkownika: od logowania/rejestracji, przez przeglądanie katalogu, po dodawanie nowego przedmiotu. Poniżej przedstawiono screenshoty każdego ekranu wraz z opisem interakcji mobilnych.

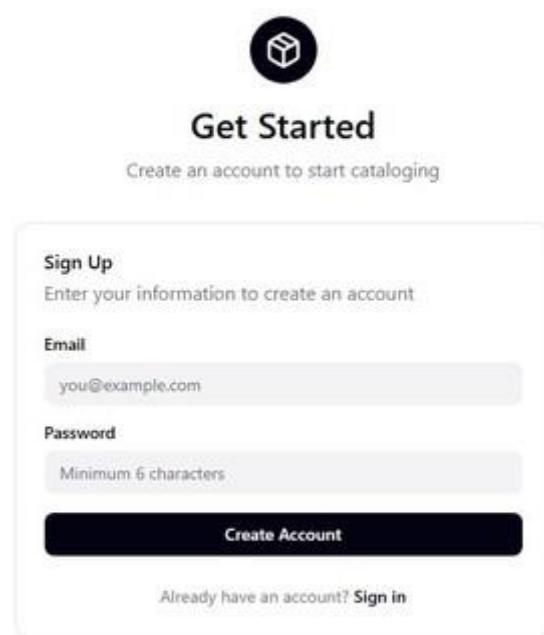
2.1 Ekran logowania (LoginScreen)



Opis ekranu: Ekran startowy dla niezalogowanego użytkownika. Wyświetla logo aplikacji (ikona pudełka), nagłówek „Welcome Back” oraz formularz z polami E-mail i Password. Przycisk „Sign In” wywołuje Firebase Auth `signInWithEmailAndPassword`. Link „Sign up” prowadzi do ekranu rejestracji.

Interakcje mobilne: Klawiatura systemowa wysuwa się automatycznie po tapnięciu pola e-mail. Pole hasła używa `PasswordVisualTransformation` (znaki maskowane). Przycisk Sign In blokuje się i pokazuje spinner podczas ładowania (`enabled = !state.isLoading`). Błąd Firebase wyświetlany pod formularzem jako czerwony tekst.

2.2 Ekran rejestracji (RegisterScreen)





Get Started

Create an account to start cataloging

Sign Up
Enter your information to create an account

Email
you@example.com

Password
Minimum 6 characters

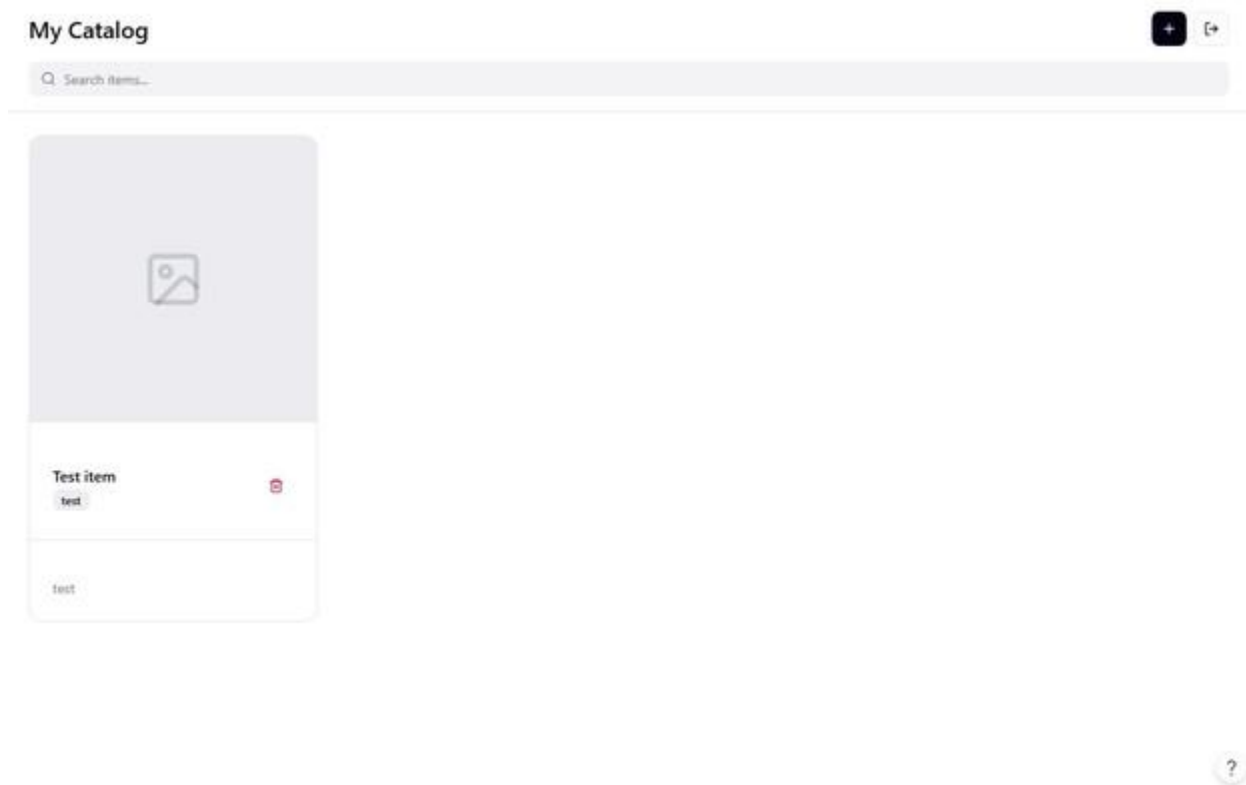
Create Account

Already have an account? [Sign in](#)

Opis ekranu: Ekran zakładania konta. Identyczny układ jak logowanie – logo, nagłówek „Get Started”, pola E-mail i Password z placeholder „Minimum 6 characters”. Przycisk „Create Account” wywołuje Firebase Auth createUserWithEmailAndPassword. Link „Sign in” prowadzi z powrotem do logowania.

Interakcje mobilne: Walidacja hasła (min. 6 znaków) realizowana lokalnie przez CatalogValidation.validatePassword() przed wywołaniem Firebase. Po pomyślnej rejestracji nawigacja do CATALOG_GRAPH z wyczyszczeniem stosu (popUpTo LOGIN inclusive=true) – użytkownik nie może cofnąć się do ekranu rejestracji.

2.3 Ekran katalogu (CatalogScreen)



Opis ekranu: Ekran główny aplikacji. TopAppBar z tytułem „My Catalog” oraz dwoma ikonami: „+” (dodaj nowy przedmiot) i wylogowanie. Pasek wyszukiwania „Search items...” filtruje katalog w czasie rzeczywistym. Karty przedmiotów wyświetlają: zdjęcie (placeholder gdy brak), nazwę, tag kategorii i opis. Ikona kosza na karcie umożliwia szybkie usunięcie.

Interakcje mobilne: Scroll pionowy listy (LazyColumn z lazy loading). Tapnięcie karty otwiera szczegóły przedmiotu (ItemDetailScreen). Wyszukiwanie realizowane lokalnie przez SearchMatcher – filtruje po nazwie, opisie i kategorii bez opóźnień sieciowych. FAB (+) w prawym dolnym rogu otwiera formularz dodawania. Dane aktualizowane automatycznie przez Firestore addSnapshotListener.

2.4 Formularz dodawania przedmiotu (ItemFormScreen)

The image shows a mobile application modal titled "Add New Item". It has a close button (X) in the top right corner. The form contains the following elements:

- Name ***: A text input field with the placeholder "Item name".
- Category**: A text input field with the placeholder "e.g., Electronics, Books, Collectibles".
- Description**: A multi-line text area with the placeholder "Add details about this item...".
- Photo**: A section with a camera icon and the text "Take Photo".
- Buttons**: "Cancel" and "Add Item" buttons at the bottom right.

Opis ekranu: Modal/ekran dodawania nowego przedmiotu. Pola: Name* (wymagane), Category (z podpowiedzią „e.g., Electronics, Books, Collectibles”), Description (wieloliniowy textarea) oraz sekcja Photo z przyciskiem „Take Photo” z ikoną aparatu. Przyciski nawigacyjne: „Cancel” i „Add Item”.

Interakcje mobilne (kluczowe dla uzasadnienia mobilności): Przycisk „Take Photo” uruchamia natywny aparat telefonu przez `ActivityResultContracts.TakePicture()` – to jedyna metoda dodania zdjęcia w aplikacji (brak wyboru z galerii). Wymaga uprawnienia `android.permission.CAMERA`. Zdjęcie jest automatycznie kompresowane (max 640px, JPEG quality 52) i enkodowane do Base64 przez `ImageUtils`. Plik tymczasowy zapisywany w `cacheDir` przez `FileProvider`. Walidacja Name (2–80 znaków) i Category (2–40 znaków) wyświetla błędy pod polami jako `supportingText`.

2.5 Przepływ nawigacyjny (User Flow)

Kompletny przepływ użytkownika oparty na Jetpack Navigation Compose:

- Start → sprawdzenie `FirebaseAuth.currentUser` → jeśli zalogowany: Katalog | jeśli nie: Logowanie
- Logowanie → [Sign In] → Katalog | [Sign up] → Rejestracja
- Rejestracja → [Create Account] → Katalog | [Sign in] → Logowanie
- Katalog → [karta] → Szczegóły | [+] → Formularz dodawania | [wyloguj] → Logowanie
- Szczegóły → [Edytuj] → Formularz edycji | [Usuń + potwierdź] → Katalog | [Wróć] → Katalog
- Formularz → [Zapisz] → Katalog | [Wróć] → poprzedni ekran (`popBackStack`)

3. User Stories z kryteriami akceptacji

Łącznie 15 user stories pokrywających 100% zaimplementowanych funkcjonalności aplikacji. Kryteria akceptacji oparte na rzeczywistym kodzie źródłowym.

US-01: Rejestracja konta

Story: Jako nowy użytkownik chcę założyć konto e-mail/hasło, aby mieć własny prywatny katalog.

Kryteria akceptacji:

- Ekran rejestracji zawiera pola: E-mail, Hasło (min. 6 znaków)
- Walidacja e-mail sprawdza format z regex `^[^\\s@]+@[^\\s@]+\\.([^\\s@]+)$`
- Walidacja hasła: minimum 6 znaków – komunikat 'Hasło musi mieć co najmniej 6 znaków.'
- Po pomyślnej rejestracji użytkownik przechodzi bezpośrednio do katalogu (LOGIN popping ze stosu)
- Błąd Firebase (np. zajęty e-mail) wyświetlany jest pod przyciskiem jako czerwony tekst
- Przycisk 'Wróć do logowania' wraca do LoginScreen

US-02: Logowanie do konta

Story: Jako zarejestrowany użytkownik chcę się zalogować e-mailem i hasłem.

Kryteria akceptacji:

- Ekran logowania: pola E-mail i Hasło z PasswordVisualTransformation
- Przycisk 'Zaloguj się' wywołuje Firebase Auth `signInWithEmailAndPassword`
- Podczas logowania wyświetlany jest `CircularProgressIndicator`, przycisk jest zablokowany (`enabled=false`)
- Błąd logowania wyświetlany jako czerwony tekst `errorMessage`
- Po pomyślnym logowaniu nawigacja do CATALOG_GRAPH, LOGIN usuwany ze stosu
- Link 'Nie masz konta? Zarejestruj się' prowadzi do RegisterScreen

US-03: Automatyczne logowanie

Story: Jako powracający użytkownik chcę nie musieć logować się przy każdym uruchomieniu.

Kryteria akceptacji:

- Przy starcie aplikacji sprawdzany jest `FirebaseAuth.getInstance().currentUser`
- Jeśli `currentUser != null` – `startDestination = CATALOG_GRAPH` (pominiecie ekranu logowania)
- Jeśli `currentUser == null` – `startDestination = LOGIN`
- Sesja utrzymywana jest przez Firebase Auth SDK automatycznie

US-04: Wylogowanie

Story: Jako użytkownik chcę się wylogować, aby moje dane były bezpieczne.

Kryteria akceptacji:

- Przycisk 'Wyloguj' widoczny w TopAppBar na ekranie CatalogScreen
- Wywołuje `viewModel.logout() → auth.signOut()` + zatrzymanie listenera Firestore
- Po wylogowaniu nawigacja do LOGIN, CATALOG_GRAPH usuwany ze stosu
- Stan `CatalogUiState` resetowany do wartości początkowych

US-05: Przeglądanie katalogu

Story: Jako użytkownik chcę widzieć listę wszystkich moich skatalogowanych przedmiotów.

Kryteria akceptacji:

- Ekran główny (CatalogScreen) wyświetla LazyColumn z kartami przedmiotów
- Każda karta zawiera: miniaturę zdjęcia (72dp), nazwę, kategorię, pierwsze 2 linie opisu (TextOverflow.Ellipsis)
- Lista sortowana malejąco po updatedAt (Query.Direction.DESENDING w Firestore)
- Dane aktualizowane w czasie rzeczywistym przez addSnapshotListener
- Gdy katalog pusty: komunikat 'Katalog jest pusty. Dodaj pierwszy przedmiot.'
- Podczas ładowania: CircularProgressIndicator

US-06: Wyszukiwanie przedmiotów

Story: Jako użytkownik chcę wyszukiwać przedmioty po tekście, aby szybko je znaleźć.

Kryteria akceptacji:

- Pole OutlinedTextField z etykietą 'Szukaj po nazwie, opisie lub kategorii' nad listą
- Wyszukiwanie realizowane lokalnie przez SearchMatcher.matches() bez opóźnień sieciowych
- SearchMatcher sprawdza pola: name, description, category (case-insensitive, lowercase Locale)
- Lista filtruje się reaktywnie przez remember(state.items, query)
- Brak wyników: komunikat 'Brak wyników wyszukiwania.'

US-07: Dodanie nowego przedmiotu

Story: Jako użytkownik chcę dodać nowy przedmiot do katalogu ze zdjęciem, nazwą, kategorią i opisem.

Kryteria akceptacji:

- FAB (+) na CatalogScreen otwiera ItemFormScreen w trybie 'Dodaj przedmiot'
- Formularz zawiera: podgląd zdjęcia (180dp), przycisk 'Zrob zdjęcie', pola Nazwa, Kategoria, Opis
- Walidacja: Nazwa 2-80 znaków, Kategoria 2-40 znaków, Opis max 500 znaków, zdjęcie max 650000 znaków base64
- Błędy walidacji wyświetlane jako supportingText pod polami (czerwony)
- Zapis do Firestore: kolekcja users/{uid}/items z polami id, name, description, category, imageBase64, createdAt, updatedAt
- Po sukcesie: navController.popBackStack()

US-08: Wykonanie zdjęcia aparatem

Story: Jako użytkownik chcę zrobić zdjęcie przedmiotu bezpośrednio aparatem telefonu.

Kryteria akceptacji:

- Przycisk 'Zrob zdjęcie' / 'Zrob nowe zdjęcie' w ItemFormScreen
- Sprawdzenie uprawnień: ContextCompat.checkSelfPermission(CAMERA)
- Jeśli brak uprawnień: wywołanie permissionLauncher.launch(Manifest.permission.CAMERA)
- Zdjęcie zapisywane do pliku tymczasowego w context.cacheDir/camera/ przez createPrivateCameraFile()
- URI przekazywane przez FileProvider (authority: packageName.fileprovider)
- Po wykonaniu: kompresja przez ImageUtils (max 640px, JPEG quality 52, base64 NO_WRAP)
- Plik tymczasowy usuwany po enkodowaniu (encodeAndDelete)
- Błąd aparatu wyświetlany jako czerwony tekst photoError

US-09: Podgląd szczegółów przedmiotu

Story: Jako użytkownik chcę zobaczyć pełne szczegóły wybranego przedmiotu.

Kryteria akceptacji:

- Dotknięcie karty na liście otwiera ItemDetailScreen
- Widok zawiera: pełne zdjęcie (240dp wysokość), nazwę (headlineSmall), kategorie (titleSmall), pełny opis
- Jeśli brak opisu: tekst 'Brak opisu'
- Przyciski 'Edytuj' i 'Usun' dostępne w wierszu Row
- Jeśli item == null (usunięty przez inny kanał): komunikat 'Nie znaleziono przedmiotu.'

US-10: Edycja przedmiotu

Story: Jako użytkownik chcę edytować dane istniejącego przedmiotu.

Kryteria akceptacji:

- Przycisk 'Edytuj' w ItemDetailScreen nawiguje do ItemFormScreen z itemId
- Formularz inicjalizowany danymi istniejącego przedmiotu (existing?.name, existing?.description, itd.)
- Tytuł TopAppBar: 'Edytuj przedmiot'
- Zapis wywołuje itemRepository.update() z nowym updatedAt = System.currentTimeMillis()
- Pola walidowane tymi samymi regułami co przy dodawaniu
- Po sukcesie: navController.popBackStack()

US-11: Usunięcie przedmiotu

Story: Jako użytkownik chcę usunąć przedmiot z katalogu.

Kryteria akceptacji:

- Przycisk 'Usun' w ItemDetailScreen wyświetla AlertDialog z pytaniem 'Usunac przedmiot?'
- Dialog zawiera tekst ostrzegawczy: 'Tej operacji nie można cofnąć.'
- Przyciski dialogu: 'Usun' (potwierdzenie) i 'Anuluj'
- Potwierdzenie wywołuje detailViewModel.deleteItem() → itemRepository.delete() w Firestore
- Podczas usuwania: przyciski Edytuj/Usun zablokowane (enabled = !deleteState.isDeleting)
- Po sukcesie: nawigacja do CATALOG z popUpTo(CATALOG) inclusive=true
- Błąd usuwania wyświetlany jako errorMessage w czerwonym kolorze

US-12: Synchronizacja z chmurą w czasie rzeczywistym

Story: Jako użytkownik chcę, aby mój katalog był automatycznie synchronizowany z Firebase.

Kryteria akceptacji:

- Dane przechowywane w Firebase Firestore: kolekcja users/{uid}/items
- Listener Firestore (addSnapshotListener) startuje przy wejściu na CatalogScreen (DisposableEffect)
- Listener zatrzymywany przy opuszczeniu CatalogScreen (onDispose)
- Zmiany z innych urządzeń/sesji widoczne natychmiast dzięki real-time updates
- CatalogViewModel utrzymywany w ramach CATALOG_GRAPH (backStackEntry scoped) – jeden ViewModel dla grafiku

US-13: Zdjęcia przechowywane jako Base64

Story: Jako użytkownik chcę, aby zdjęcia były przechowywane w bazie danych bez osobnego serwera plików.

Kryteria akceptacji:

- Zdjęcia kompresowane do max 640x640px, JPEG quality 52

- Enkodowane jako Base64 (NO_WRAP) i przechowywane w polu imageBase64 dokumentu Firestore
- Limit rozmiaru: 650000 znaków base64 (~488KB)
- Komponent Base64Image dekoduje ciąg i wyświetla przez BitmapFactory.decodeByteArray
- Plik tymczasowy w cacheDir usuwany natychmiast po enkodowaniu

US-14: Walidacja formularza

Story: Jako użytkownik chcę widzieć czytelne komunikaty błędów przy nieprawidłowych danych.

Kryteria akceptacji:

- Walidacja realizowana przez CatalogValidation.validateItem() przed zapisem do Firestore
- Nazwa: brak lub poza zakresem 2-80 znaków → 'Nazwa musi mieć od 2 do 80 znaków.'
- Kategoria: brak lub poza zakresem 2-40 znaków → 'Kategoria musi mieć od 2 do 40 znaków.'
- Opis: powyżej 500 znaków → 'Opis może mieć maksymalnie 500 znaków.'
- Zbyt duże zdjęcie → 'Zdjęcie jest zbyt duże. Wykonaj zdjęcie ponownie.'
- Błędy wyświetlane jako isError=true + supportingText pod odpowiednim polem OutlinedTextField
- Błędy czyszczone przy kolejnym wywołaniu saveItem()

US-15: Nawigacja między ekranami

Story: Jako użytkownik chcę sprawnie poruszać się między ekranami aplikacji.

Kryteria akceptacji:

- Nawigacja oparta na Jetpack Navigation Compose z NavHost
- Trasy: login, register, catalog_graph (nested: catalog, detail/{itemId}, form?itemId={itemId})
- Przycisk 'Wróć' w ItemFormScreen i ItemDetailScreen (IconButton w navigationIcon)
- Wylogowanie i rejestracja używają popUpTo z inclusive=true aby czyścić stos
- CatalogViewModel współdzielony między catalog, detail i form przez backStackEntry scoped do catalog_graph
- Parametr itemId w FORM jest nullable – null = tryb dodawania, non-null = tryb edycji

4. MVP vs Funkcje Dodatkowe

Tabela oparta na analizie kodu źródłowego repozytorium. Kolumna 'Status' odzwierciedla rzeczywisty stan implementacji w v1.0.

Funkcja	Status	Priorytet	Uwagi
Rejestracja konta (e-mail/hasło)	ZREALIZOWANE (MVP)	Krytyczny	Firebase Auth
Logowanie / auto-logowanie	ZREALIZOWANE (MVP)	Krytyczny	FirebaseAuth.currentUser
Wylogowanie	ZREALIZOWANE (MVP)	Wysoki	auth.signOut()
Przegląd listy przedmiotów	ZREALIZOWANE (MVP)	Krytyczny	LazyColumn + Firestore listener
Wyszukiwanie (nazwa/opis/kategoria)	ZREALIZOWANE (MVP)	Wysoki	SearchMatcher, lokalne
Dodanie przedmiotu	ZREALIZOWANE (MVP)	Krytyczny	ItemFormScreen, Firestore create
Zdjęcie aparatem (CAMERA)	ZREALIZOWANE (MVP)	Krytyczny	ActivityResultContracts.TakePicture
Zdjęcia jako Base64 w Firestore	ZREALIZOWANE (MVP)	Wysoki	ImageUtils, max 640px, q52
Edycja przedmiotu	ZREALIZOWANE (MVP)	Wysoki	Firestore update, updatedAt
Usuwanie z potwierdzeniem	ZREALIZOWANE (MVP)	Wysoki	AlertDialog, Firestore delete
Widok szczegółów przedmiotu	ZREALIZOWANE (MVP)	Wysoki	ItemDetailScreen
Synchronizacja real-time	ZREALIZOWANE (MVP)	Wysoki	addSnapshotListener Firestore
Walidacja formularza	ZREALIZOWANE (MVP)	Wysoki	CatalogValidation
Wybor zdjęcia z galerii	NIE ZREALIZOWANE	Sredni	Brak ActivityResultContracts.GetContent
Filtrowanie po kategorii (chip)	NIE ZREALIZOWANE	Sredni	Brak – tylko wyszukiwanie tekstowe
Sortowanie listy	NIE ZREALIZOWANE	Niski	Stale: malejaco po updatedAt
Tryb offline (cache lokalny)	NIE ZREALIZOWANE	Sredni	Firestore offline cache nieaktywowany

Skanowanie kodów kreskowych	NIE ZREALIZOWANE	Niski	Brak w kodzie
Eksport CSV/PDF	NIE ZREALIZOWANE	Niski	Brak w kodzie
Udostępnianie przez share sheet	NIE ZREALIZOWANE	Niski	Brak Intent.ACTION_SEND
Powiadomienia push	NIE ZREALIZOWANE	Niski	Brak Firebase Messaging

3.1 Zakres MVP (zrealizowany)

MVP obejmuje pełny cykl życia przedmiotu z autentykacją: rejestracja → logowanie → dodanie zdjęcia aparatem → opis/kategoria → przeglądanie z wyszukiwaniem → edycja → usunięcie. Backend oparty na Firebase Auth + Firestore z real-time synchronizacją.

3.2 Funkcje poza MVP (niezrealizowane, potencjalna roadmapa)

1. v1.1 – Wybór zdjęcia z galerii (ActivityResultContracts.GetContent)
2. v1.2 – Filtry kategorii jako chipy (FilterChip) nad listą
3. v1.3 – Sortowanie listy (dropdown: data, nazwa A-Z, kategoria)
4. v1.4 – Aktywacja Firestore offline cache (Firestore.setFirestoreSettings z persistence)
5. v2.0 – Skaner kodów kreskowych (ML Kit Barcode Scanning)
6. v2.1 – Eksport katalogu do CSV/PDF

5. Opis Aplikacji do Google Play

4.1 Krótki opis (do 80 znaków)

Fotografuj i kataloguj swoje rzeczy. Szybko, prosto, z chmurą.

4.2 Pełny opis

Przedmioty to mobilna aplikacja do katalogowania przedmiotów, która działa na Twoim telefonie z aparatem i dostępem do internetu.

Co możesz robić:

- Fotografuj przedmioty bezpośrednio aparatem i dodaj je do katalogu w kilka sekund
- Opisuj każdy przedmiot – nazwa, kategoria i dowolny opis
- Przeglądaj i wyszukuj po nazwie, opisie lub kategorii
- Edytuj i usuwaj wpisy w dowolnej chwili
- Twój katalog synchronizuje się z chmurą – dostępny z każdego urządzenia po zalogowaniu

Dla kogo:

- Do katalogowania wyposażenia domu – piwnicy, garażu, warsztatu
- Do inwentaryzacji kolekcji (książki, elektronika, narzędzia)
- Dla każdego, kto chce wiedzieć, co posiada i gdzie to jest

Technologie: Firebase Auth, Firestore, Jetpack Compose, Material3, Android Camera.

4.3 Słowa kluczowe (ASO)

katalog przedmiotów, inwentaryzacja domowa, organizacja rzeczy, zarządzanie kolekcją, spis przedmiotów, home inventory, item catalog, zdjęcie przedmiotu, Firebase katalog, Kotlin Android

4.4 Dane techniczne

Parametr	Wartość
Nazwa aplikacji	Przedmioty
Package ID	pl.edu.przedmioty
Wersja	1.0 (versionCode 1)
Minimalna wersja Androida	Android 6.0 (API 23)
Target SDK	Android 16 (API 36)
Język kodu	Kotlin 100%
Wymagane uprawnienia	INTERNET, CAMERA
Backend	Firebase Auth + Firestore
UI Framework	Jetpack Compose + Material3

6. Checklista Testów Akceptacyjnych

Instrukcja: Status wypełnij przed release: ✓ Zaliczony / X Niezaliczony / — Pominięty. Testy TA-01 do TA-22 obowiązkowe dla v1.0.

ID	Obszar	Opis testu	Oczekiwany wynik	Status
TA-01	Auth	Rejestracja z poprawnym e-mailem i hasłem 6+ znaków	Konto utworzone, przejście do katalogu	✓
TA-02	Auth	Rejestracja z hasłem krótszym niż 6 znaków	Komunikat 'Hasło musi mieć co najmniej 6 znaków.'	✓
TA-03	Auth	Rejestracja z błędnym formatem e-mail	Komunikat 'Wpisz poprawny adres e-mail.'	✓
TA-04	Auth	Logowanie z poprawnymi danymi	Przejście do CatalogScreen	✓
TA-05	Auth	Logowanie z błędnym hasłem	Czerwony komunikat błędu Firebase	✓
TA-06	Auth	Uruchomienie aplikacji po wcześniejszym zalogowaniu	Pominięcie ekranu logowania, startDestination = catalog_graph	✓
TA-07	Auth	Wylogowanie	Powrót do LoginScreen, stos nawigacji wyczyszczony	✓
TA-08	Katalog	Dodanie przedmiotu z nazwą, kategorią, opisem i zdjęciem	Przedmiot pojawia się na liście, posortowany najnowszy pierwszy	✓
TA-09	Katalog	Dodanie bez zdjęcia (pole imageBase64 puste)	Błąd walidacji – zdjęcie jest wymagane (image validation)	✓

TA-10	Katalog	Dodanie z nazwa ponizej 2 lub powyzej 80 znakow	Komunikat 'Nazwa musi miec od 2 do 80 znakow.'	✓
TA-11	Katalog	Dodanie z kategoria ponizej 2 lub powyzej 40 znakow	Komunikat 'Kategoria musi miec od 2 do 40 znakow.'	✓
TA-12	Katalog	Dodanie z opisem dluzszym niz 500 znakow	Komunikat 'Opis moze miec maksymalnie 500 znakow.'	✓
TA-13	Wyszukiwanie	Wpisanie frazy z nazwy przedmiotu	Lista filtruje sie na biezaco (SearchMatcher)	✓
TA-14	Wyszukiwanie	Wpisanie frazy z opisu lub kategorii	Przedmioty z pasujacymi polami sa widoczne	✓
TA-15	Wyszukiwanie	Wpisanie frazy bez dopasowania	Komunikat 'Brak wynikow wyszukiwania.'	✓
TA-16	Szczegoly	Otwarcie przedmiotu z listy	ItemDetailScreen z pelnym zdjeciem (240dp), nazwa, kategoria, opis	✓
TA-17	Edycja	Edycja nazwy istniejacego przedmiotu	Nowa nazwa widoczna na liscie, updatedAt zaktualizowane	✓
TA-18	Usuwanie	Usuniecie z potwierdzeniem w AlertDialog	Przedmiot znika z listy, powrot do CatalogScreen	✓
TA-19	Usuwanie	Anulowanie usuwania w AlertDialog	Przedmiot pozostaje na liscie	✓
TA-20	Aparat	Przycisk 'Zrob zdjecie' bez uprawnien aparatu	System pyta o uprawnienie CAMERA	✓
TA-21	Aparat	Odmowa uprawnien aparatu	Komunikat 'Brak zgody na uzycie aparatu.'	✓

TA-22	Aparat	Wykonanie zdjęcia i powrót do formularza	Miniatura zdjęcia wyświetlona (180dp), przycisk zmienia tekst na 'Zrob nowe zdjęcie'	✓
TA-23	Real-time	Dodanie przedmiotu na innym urządzeniu (ta sama sesja Firebase)	Przedmiot pojawia się na liście bez odświeżania	✓
TA-24	Nawigacja	Przycisk Wróć w ItemFormScreen	Powrót do poprzedniego ekranu bez zapisu	✓
TA-25	Uprawnienia	Aplikacja bez uprawnień CAMERA przy starcie	Aplikacja uruchamia się poprawnie, błąd pojawia się tylko przy próbie zdjęcia	✓

5.1 Kryteria zaliczenia release v1.0

Wymagane zaliczenie WSZYSTKICH testów TA-01 do TA-22. Testy TA-23 (real-time multi-device) i TA-25 opcjonalne dla v1.0.

7. Changelog

Historia zmian na podstawie commitów z repozytorium GitHub (henick408/projekt-mobilne).
Autorzy: Henick = Marcel Snopkowski (Backend), Rabarbara2 = Agata Mróz (Frontend).

v1.0.0 – 2026-06-07/08 (aktualna, premierowa)

07.06.2026 – Setup i fundamenty (Marcel)

- [feat] Initial commit – inicjalizacja projektu Android
- [feat] Zależności – konfiguracja build.gradle.kts (Compose, Firebase, Navigation)
- [feat] Firebase – integracja Firebase Auth + Firestore, google-services.json
- [feat] Modele – data class CatalogItem, ItemDraft
- [feat] Repo do autoryzacji – AuthRepository (login, register, logout)
- [feat] Repo do przedmiotów – ItemRepository (create, update, delete, listen)
- [feat] Jeden wspólny AppViewModel – współdzielony ViewModel w CATALOG_GRAPH
- [feat] ViewModel do ogarniania usuwania przedmiotów – ItemDetailViewModel
- [refactor] Rozdzielenie ViewModeli – CatalogViewModel, ItemFormViewModel, LoginViewModel, RegisterViewModel
- [feat] ViewModel do formularza – ItemFormViewModel z walidacją CatalogValidation

07.06.2026 – Ekran i nawigacja (Agata)

- [feat] Nawigacja i puste screeny – NavHost z trasami login/register/catalog/detail/form
- [feat] Logowanie – LoginScreen z Firebase Auth signInWithEmailAndPassword
- [feat] Rejestracja – RegisterScreen z Firebase Auth createUserWithEmailAndPassword
- [feat] Ekran z przedmiotami i niby zdjęciem – CatalogScreen z LazyColumn, Base64Image, FAB
- [feat] Zmiany w nawigacji, placeholderzy do szczegółów i formularza – trasy detail/{itemId}, form?itemId
- [feat] Szczegóły przedmiotu – ItemDetailScreen z AlertDialog potwierdzenia usunięcia
- [feat] Formularz przedmiotu – ItemFormScreen (dodawanie i edycja) z walidacją pól
- [feat] Wyszukiwanie przedmiotu – SearchMatcher (case-insensitive, pola: name/description/category)

07.06.2026 – Integracja aparatu (Marcel)

- [feat] Obsługa zdjęć (bez aparatu) – Base64Image komponent, dekodowanie i wyświetlanie zdjęć
- [feat] Obsługa aparatu – ActivityResultContracts.TakePicture, FileProvider, runtime permission CAMERA

08.06.2026 – Testy, fixes i polityka prywatności (Marcel)

- [test] Jakies testy – testy jednostkowe (ViewModels, SearchMatcher, CatalogValidation)
- [fix] Pionowe zdjęcia są w końcu pionowe i da się przybliżyć zdjęcie – ImageUtils: fix orientacji EXIF, pinch-to-zoom w ItemDetailScreen
- [docs] Polityka prywatności – dokument wymagany przez Google Play Store

v1.1.0 – planowana

- [feat] Wybór zdjęcia z galerii (ActivityResultContracts.GetContent)
- [feat] Filtry kategorii jako chipy nad listą przedmiotów
- [feat] Sortowanie listy (data dodania, nazwa A-Z, kategoria)
- [feat] Aktywacja Firestore offline cache (tryb bez internetu)

8. Instrukcja Użytkownika

7.1 Pierwsze uruchomienie

7. Zainstaluj aplikację na urządzeniu z Androidem 6.0+
8. Uruchom aplikację – pojawi się ekran 'Katalog Przedmiotów'
9. Naciśnij 'Nie masz konta? Zarejestruj się' aby założyć konto
10. Podaj adres e-mail i hasło (min. 6 znaków), naciśnij 'Utwórz konto'
11. Zostaniesz automatycznie przeniesiony do pustego katalogu

7.2 Dodawanie przedmiotu

12. Na ekranie głównym naciśnij przycisk + (prawy dolny róg)
13. Naciśnij 'Zrób zdjęcie' – aplikacja poprosi o uprawnienie do aparatu
14. Wykonaj zdjęcie przedmiotu aparatem telefonu
15. Wróć do formularza – miniatura zdjęcia pojawi się u góry
16. Wypełnij pola: Nazwa (wymagana, 2–80 znaków), Kategoria (wymagana, 2–40 znaków), Opis (opcjonalny, max 500 znaków)
17. Naciśnij 'Zapisz' – przedmiot pojawi się na liście

7.3 Przeglądanie i wyszukiwanie

- Na ekranie głównym widoczna jest lista wszystkich przedmiotów, posortowana od najnowszego
- Skorzystaj z pola 'Szukaj po nazwie, opisie lub kategorii' – wyniki filtrują się na bieżąco
- Dotknij kartę przedmiotu, aby zobaczyć pełne szczegóły

7.4 Edycja i usuwanie

- W widoku szczegółów naciśnij 'Edytuj' – formularz wypełni się aktualnymi danymi
- Wprowadź zmiany i naciśnij 'Zapisz'
- Aby usunąć przedmiot, naciśnij 'Usuń' – potwierdź w dialogu. Uwaga: operacji nie można cofnąć

7.5 Logowanie i konto

- Twoje dane zapisane są w chmurze Firebase – zaloguj się na innym urządzeniu tym samym e-mailem
- Wyloguj się przyciskiem 'Wyloguj' w prawym górnym rogu ekranu głównego
- Hasło musi mieć co najmniej 6 znaków

7.6 FAQ

Aplikacja nie uruchamia aparatu – co robić?

Przejdź do Ustawienia systemowe > Aplikacje > Przedmioty > Uprawnienia > Aparat i zezwól na dostęp.

Czy dane są bezpieczne?

Dane przechowywane są w Firebase Firestore z regułami bezpieczeństwa – każdy użytkownik widzi wyłącznie swoje przedmioty (kolekcja users/{uid}/items). Zdjęcia przechowywane jako Base64 w tej samej bazie danych.

Aplikacja nie łączy przedmiotów – co robić?

Sprawdź połączenie z internetem. Aplikacja wymaga aktywnego połączenia do synchronizacji z Firestore. Tryb offline nie jest aktualnie obsługiwany.

Czy mogę wybrać zdjęcie z galerii?

Nie – w aktualnej wersji (v1.0) jedyną metodą dodania zdjęcia jest aparat telefonu. Wybór z galerii planowany jest w v1.1.